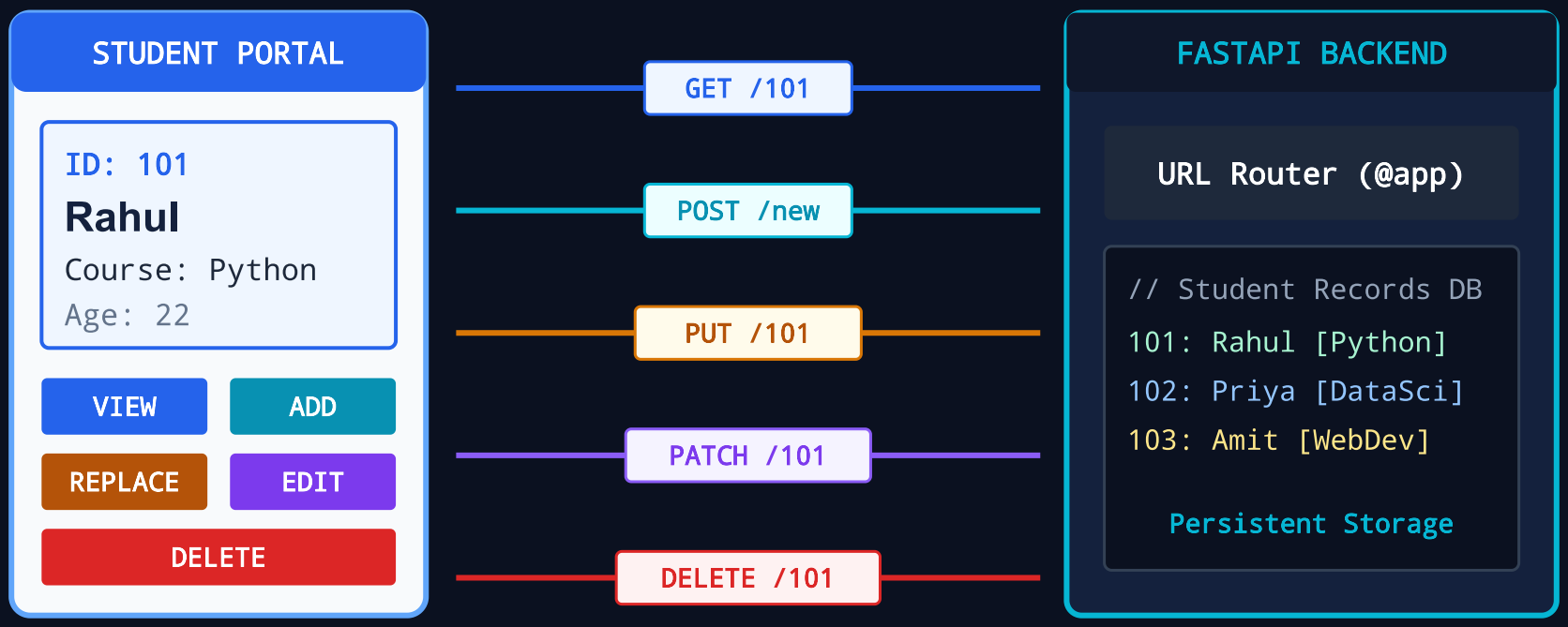
BACKEND API FUNDAMENTALS

HTTP METHODS EXPLAINED

> “How APIs read, create, update, and delete data.”

GET POST PUT PATCH DELETE

FIG 04.1 // STUDENT MANAGEMENT API TOPOLOGY — 5 CRUD OPERATIONS



CORE TEACHING EXAMPLE One consistent Student Management API across all slides

01 // COMMUNICATING INTENT

An API needs to know what you want to do.

When a client sends an HTTP request, the **HTTP method** indicates the intended action. Different HTTP methods communicate different intentions for the same resource.

TARGET RESOURCE: /students/101

5 DISTINCT INTENDED ACTIONS

```
// CURRENT RECORD
{
  "id": 101,
  "name": "Rahul",
  "course": "Python",
  "age": 22
}
```

- View a student — GET
- Add a student — POST
- Replace student details — PUT
- Modify selected information — PATCH
- Delete a student record — DELETE

METHOD	BASIC PURPOSE
GET	Read or retrieve information
POST	Create a new resource or submit data
PUT	Replace the entire resource representation
PATCH	Partially modify selected fields of a resource
DELETE	Request removal of the identified resource

> **Same URL + Different HTTP Method = Different Action.**

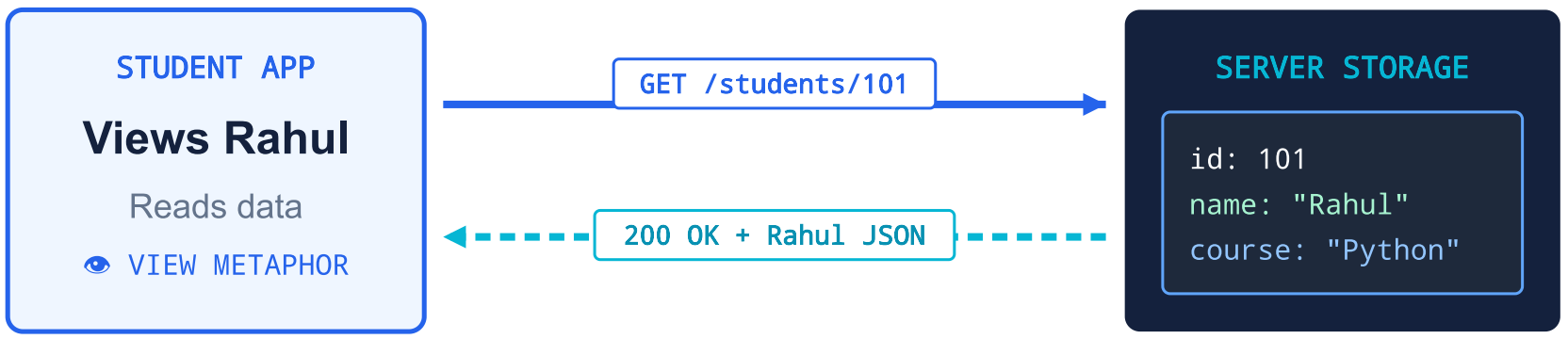
02 // RETRIEVE INFORMATION

GET — I want to see the data.

Scenario: You want to view Rahul's student record.

```
// CLIENT REQUEST
GET /students/101
```

FIG 04.3 // GET OPERATION (SAFE READ) — RETRIEVE ONLY




```
// SERVER RESPONSE (200 OK)
{
  "id": 101,
  "name": "Rahul",
  "course": "Python",
  "age": 22
}
```

> **GET retrieves data. It is not intended to modify the resource.**

03 // SUBMIT & CREATE

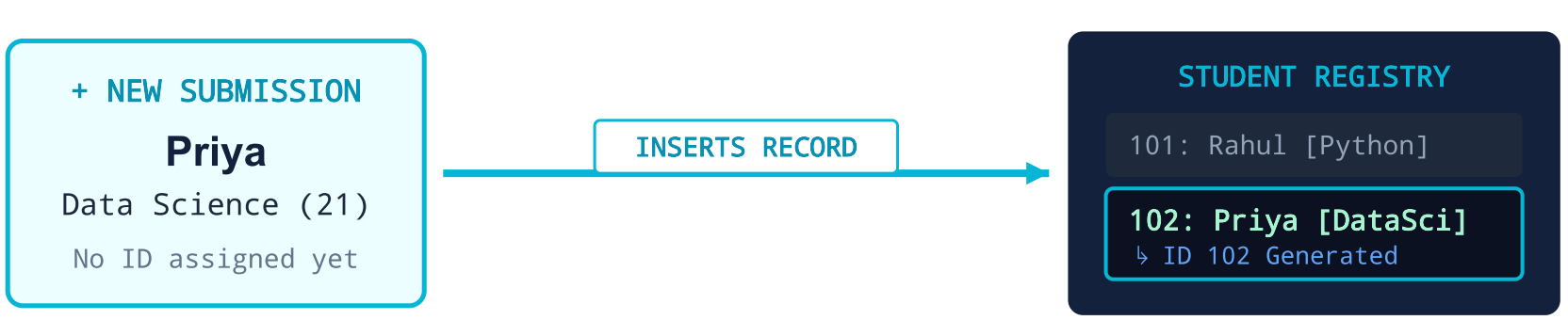
POST — I want to add something new.

Scenario: A new student joins the course: **Priya**, Data Science, Age 21.

```
// CLIENT REQUEST (POST /students)                    INCLUDES REQUEST BODY
POST /students
{
  "name": "Priya",
  "course": "Data Science",
  "age": 21
}
```

FIG 04.4 // POST OPERATION (RESOURCE CREATION)

NEW ID ASSIGNED BY SERVER




```
// SERVER RESPONSE (201 Created)
{
  "id": 102,
  "name": "Priya",
  "course": "Data Science",
  "age": 21
}
```

POST submits data and is commonly used to create a new resource.

04 // COMPLETE REPLACEMENT

PUT — Replace the existing information.

Scenario: You want to replace Rahul's record representation completely.

ORIGINAL RECORD (101)

```
{
  "id": 101,
  "name": "Rahul",
  "course": "Python",
  "age": 22
}
```

SUBMITTED PUT BODY

```
{
  "name": "Rahul",
  "course": "Machine Learning",
  "age": 23
}
```

FIG 04.5 // PUT REPLACEMENT SEMANTICS

ENTIRE REPRESENTATION REPLACED



TECHNICAL SPECIFICATION (RFC 9110)

A PUT request sends the **complete representation** that should replace the existing resource. The course and age both change in one replacement operation.

PUT = Replace the resource representation.

05 // PARTIAL MODIFICATION

PATCH — I only want to change one thing.

Scenario: Rahul changes only his course to Machine Learning. His age (22) and name remain unchanged.

// PATCH REQUEST (ONLY MODIFIED FIELD SENT) — PARTIAL BODY

```
PATCH /students/101
{
  "course": "Machine Learning"
}
```

FIG 04.6 // PATCH TARGETED FIELD MODIFICATION — UNTOUCHED FIELDS PRESERVED



// UPDATED RECORD (AGE 22 PRESERVED)

```
{
  "id": 101,
  "name": "Rahul",
  "course": "Machine Learning",
  "age": 22
}
```

PATCH = Modify selected information, not necessarily the entire resource.

06 // KEY ARCHITECTURAL DISTINCTION

PUT and PATCH are NOT the same.

While both update data, their intended semantics are fundamentally different:

PUT

REPLACE

- Replaces the entire resource representation.
- Generally sends the complete updated object.
- **Think:** Replace the whole document.

```
Old record ➔ Complete Replacement
➔ New representation
```

PATCH

MODIFY PARTIAL

- Applies partial modifications to a resource.
- Sends only the fields being changed.
- **Think:** Edit specific fields.

```
Old record ➔ Modify Selected Field
➔ Updated representation
```

WARNING FOR BACKEND DEVELOPERS

Do not assume PUT and PATCH are interchangeable. Using PUT when you only send one field can accidentally overwrite or erase other fields if the API strictly implements RFC replacement!

Optional Technical Note: PUT is defined as an *idempotent* method (repeating it produces the same state). PATCH is not guaranteed to be idempotent; that depends on the specific patch instructions.

07 // REQUEST REMOVAL

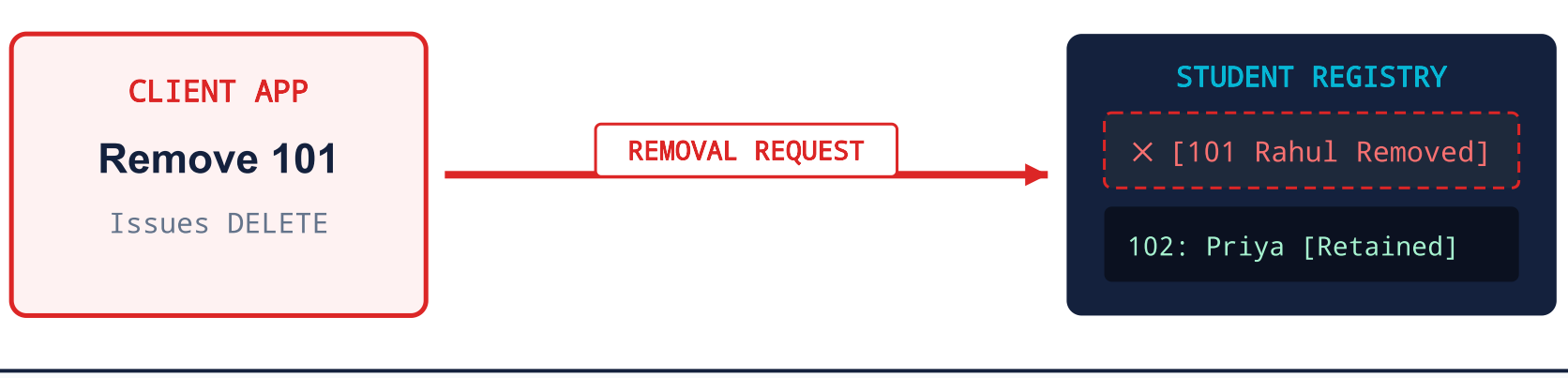
DELETE — I want to remove this record.

Scenario: Rahul finishes his degree and his student record needs to be removed.

```
// CLIENT DELETION REQUEST
DELETE /students/101
```

FIG 04.8 // CONTROLLED RECORD REMOVAL

REMOVAL PROCESSED BY SERVER



200 OK

Deletion succeeded and returns a confirmation message or representation of the deleted student.

204 No Content

Deletion succeeded with no message body returned. The resource is cleanly gone.

DELETE = Request removal of a resource.

08 // COMPLETE METHOD MATRIX

One student record. Five different operations.

```
// TARGET RESOURCE: STUDENT 101
{"id": 101, "name": "Rahul", "course": "Python",
"age": 22}
```

BASELINE RECORD

GET	/students/101	**View Rahul's record** without modifying anything.
POST	/students	**Add a new student** (e.g. Priya) into the system.
PUT	/students/101	**Replace Rahul's record** completely with new submitted data.
PATCH	/students/101	**Change only Rahul's course** while preserving his age and name.
DELETE	/students/101	**Remove Rahul's record** permanently from the registry.

The URL identifies the student. The HTTP method determines the action.

10 // KNOWLEDGE CHECK

Remember these five HTTP methods.

METHOD	WHAT IT DOES
GET	Retrieve data without modifying the server state
POST	Submit data, commonly to create a new resource
PUT	Replace an entire existing resource representation
PATCH	Partially modify specific fields of a resource
DELETE	Request removal of the identified resource

THE 5-SECOND CHEAT SHEET

GET it. POST it. PUT a replacement. PATCH a part. DELETE it.

“Understanding HTTP methods is an important step toward building your first API.”

COMING NEXT // PART 05

Introduction to FastAPI: Your First Python API

Save this carousel and follow the series to learn FastAPI step by step.